Experiment – 1

1. Title: - Download, Install Android Studio and Run Hello World Application.

2. Objective: -

- To download and install Android Studio IDE on the system.
- To set up the Android development environment.
- To create and run a basic "Hello World" Android application.
- To understand the basic structure of an Android project.
- To familiarize with the Android Studio interface and tools.

3. Theory: - Android Studio is the official Integrated Development Environment (IDE) for Android application development, based on IntelliJ IDEA. It provides comprehensive tools for building Android applications including code editor, debugger, performance tools, and a flexible build system.

4. Android Application Structure:

- **Activities:** Represent a single screen with user interface
- **Layouts:** XML files that define the UI structure
- **Manifest File:** Contains essential application information
- **Resources:** Images, strings, colors, and other assets
- **Java/Kotlin Files:** Application logic and functionality

5. Requirements: -

Hardware Requirements:

- **RAM:** Minimum 8 GB (16 GB recommended)
- **Storage:** Minimum 4 GB free space
- **Processor:** Intel i5 or equivalent
- **Operating System:** Windows 10/11, macOS 10.14+, or Linux

Software Requirements:

- Java Development Kit (JDK) 8 or higher
- Android Studio (Latest version)
- Android SDK
- Internet connection for downloading components

System Configuration:

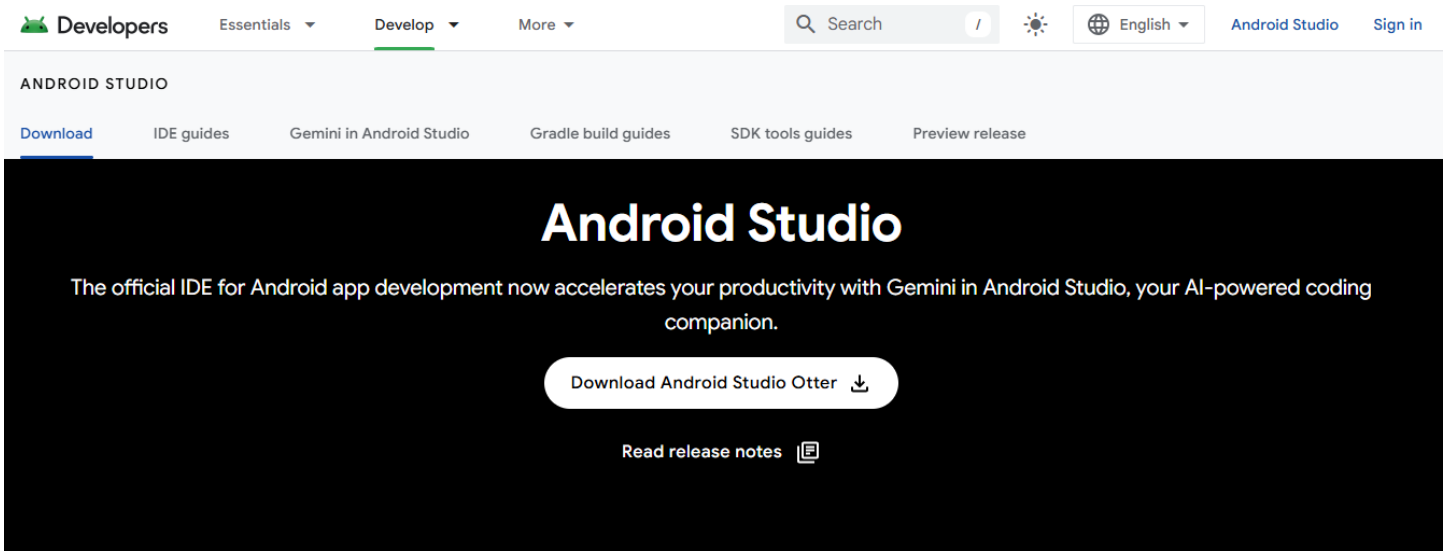
- Enable hardware acceleration for emulator
- Configure environment variables (JAVA_HOME, ANDROID_HOME)
- Sufficient system resources for smooth development

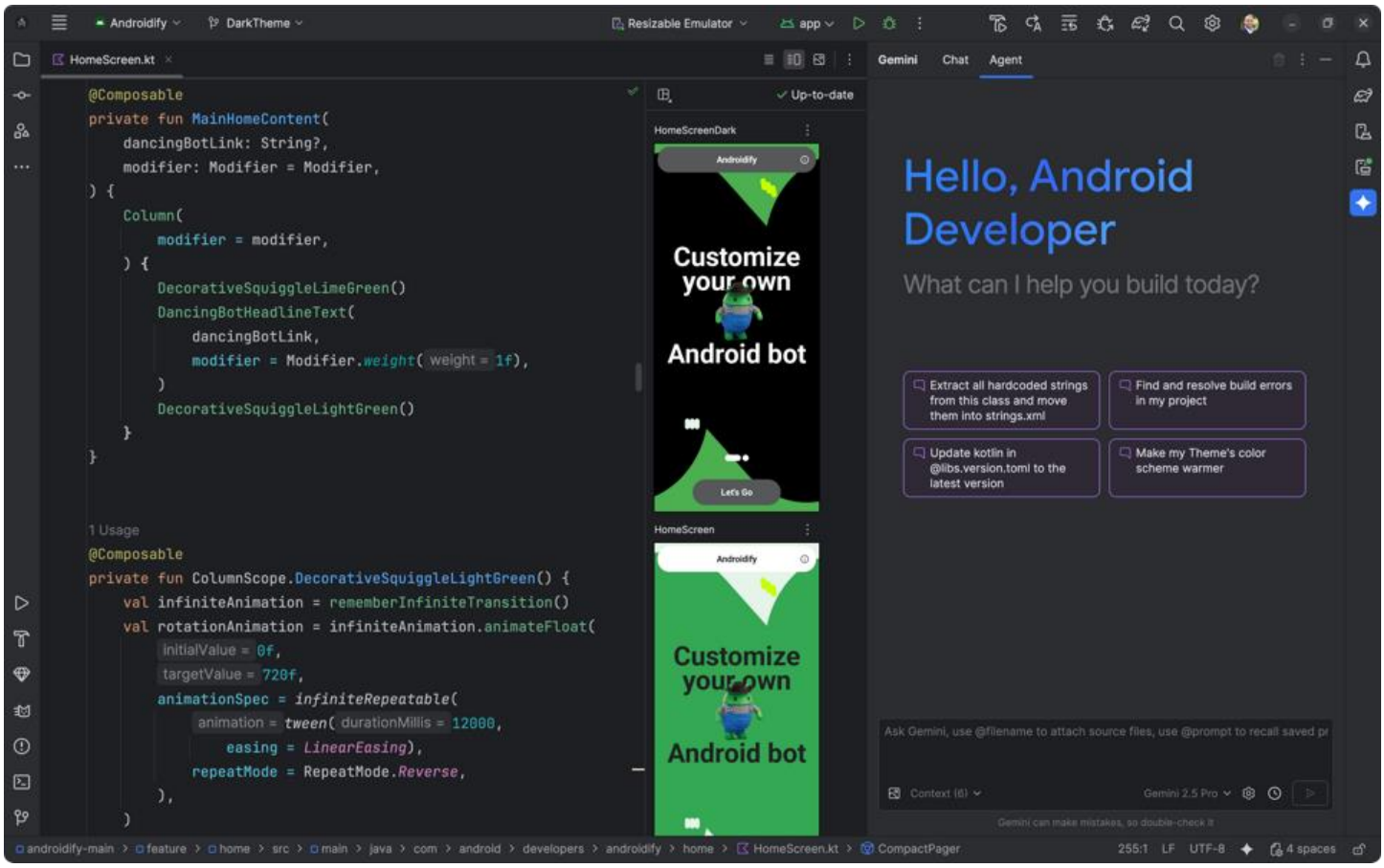
6. Steps to Download and Install Android Studio

Step 1: Download Android Studio

1. **Open a web browser** such as Chrome, Edge, or Firefox, and visit the official **Android Studio download page** at https://developer.android.com/studio.

This is the official website of Android Developers**,** where the latest version of Android Studio is available. The page will automatically recognize your operating system (Windows, macOS, or Linux) and display the appropriate download option.





2. **Click on "Download Android Studio."**
   This will open the Terms and Conditions page containing the Android Studio License Agreement**.**
3. **Review the License Agreement.**
   Read through the agreement to understand the terms of use.
4. **Accept the Terms and Conditions.**
   At the bottom of the page, check the box labeled "I have read and agree with the above terms and conditions."
5. **Start the Download.**
   Click the "Download Android Studio" button again to begin downloading the setup file.
6. **Choose a Save Location.**
   When prompted, save the installer file to an easy-to-find location, such as your **Downloads** folder or Desktop**.**
7. **Wait for the Download to Finish.**
   The download may take several minutes, depending on your internet speed. Once completed, you'll have the Android Studio installer file ready for installation.

Step 2: Install Android Studio

1. Open the folder where you saved the downloaded Android Studio setup file (for example, in your Downloads folder).
2. Double-click the installer file to start the installation process.
3. If a User Account Control (UAC) dialog appears asking for permission to allow the installer to make changes to your computer, click "Yes" to continue.

The Welcome to Android Studio Setup dialog displays.

1. **Click "Next"** to begin the Android Studio installation process using the setup wizard.
2. **Use the Recommended Settings.**
   Keep the default installation options (such as components and installation path) unless you have specific requirements, then click **Next** to continue.
3. **Finish Installation.**
   Once the installation process is complete, click **"Finish"** to launch Android Studio.
4. **Select a Theme.**
   When Android Studio opens for the first time, you'll be asked to choose a **theme** — either **Light** or **Dark** mode.
   (Most guides use the Light theme, but you can choose whichever you prefer.)
5. **Complete Setup.**
   Android Studio may download additional components (like SDK files). Wait for these to finish, and then click **"Finish"** to complete the setup.

Step 3: First-Time Setup

- **Initial Setup and Configuration**

i. Launch Android Studio:
After installation, open **Android Studio** from your desktop shortcut or Start menu.

ii. Setup Wizard:

- Select the **"Standard"** installation type (recommended for most users).
- Choose your preferred **UI theme** — *Light* or *Dark*.
- Review the configuration summary, then click **"Finish"** to proceed.

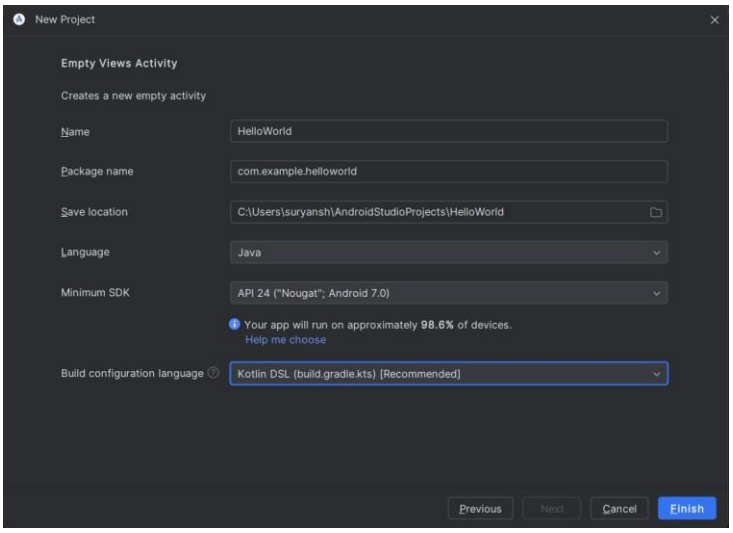
iii. SDK Components Download:

- Android Studio will automatically **download the required SDK components**.
- This process may take **10–30 minutes**, depending on your internet speed.
- Components include:
  - **Android SDK**
  - **Android SDK Platform**
  - **Android Virtual Device (AVD)**

Once the setup completes, the **“Welcome to Android Studio”** window will appear — now you’re ready to start creating your first Android app!

Step 4: Verify Installation

- **Create a New Project:**
  - On the Welcome to Android Studio screen, click **“New Project”**.
  - Select **“Empty Activity”** (recommended for beginners).
  - Click **Next** to continue.
- **Configure Your Project:**
  - Enter your **Application Name**
  - Specify a **Package Name**
  - Choose a **Save Location** on your computer where the project files will be stored.
  - Set the **Language** to *Java* or *Kotlin*
  - Select the **Minimum SDK** (*API 24: Android 7.0 (Nougat)* or higher).
  - Click **Finish** to create your project.
- **Verify the Setup:**
  - Wait for Android Studio to build and load the project.
  - If you see the project workspace and no error messages, your installation is complete and working properly!
  - Select API level (API 21 or higher recommended



- **Wait for Project Setup:**
  - Android Studio will automatically **run Gradle sync** and **download necessary dependencies** for your new project.
  - This process may take a few minutes, especially during the first setup.

- **Test the Installation:**
  - Open the **AVD Manager** (Android Virtual Device Manager) and **create a new virtual device** for example, a Pixel 8 emulator.
  - Click **Run ▶** to build and launch the default **"Hello World"** app.
  - If the emulator opens and the app runs successfully, your **Android Studio installation and setup are complete!** □

7. Code Snippet (Run Hello World App)

AndroidMainfest.xml:-

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.helloworld">

    <application
        android:allowBackup="true"
        android:label="Hello World Application"
        android:icon="@mipmap/ic_launcher"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.AppCompat.Light.NoActionBar">

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
    </application>
```

MainActivity.java

```
package com.example.helloworld;

import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:background="#F5F5F5">
```

8. Result/Output: -

After successfully downloading and installing Android Studio, a new Android project was created. Upon running the project on an emulator or connected device, the application launched without errors, and the text “Hello World!” was displayed clearly on the screen. This confirms that the IDE, SDK, and emulator are properly set up and functional.

9. Conclusion: -

In this tutorial, I showed you how to install Android Studio and create your first app: Hello World, the simplest Android app. After following this tutorial to create your first Android app, you are on your way to a promising career in developing apps!.

Experiment – 2

1. Aim: - To develop a Simple Android Application that uses Layout Managers

2. Objective: -

- To understand the concept of layout managers in Android.
- To design a simple Android application using different layout types such as Linear Layout, Relative Layout, and Constraint Layout.
- To learn how to arrange UI components (buttons, text views, etc.) properly using layout managers

3. Theory: -

In Android, a Layout Manager is used to control the arrangement of user interface elements on the screen. It determines how components like buttons, text views, and images are positioned and aligned within an activity. LinearLayout arranges elements in a single row or column, making it suitable for simple designs. RelativeLayout allows elements to be positioned relative to each other or the parent, providing more flexibility. ConstraintLayout offers advanced control, enabling complex and responsive layouts by defining constraints between elements. Using layout managers effectively helps create organized, visually appealing, and adaptable Android applications across different screen sizes and orientations.

4. Requirements: -

Hardware Requirements:

- RAM: Minimum 8 GB (16 GB recommended)
- Storage: Minimum 4 GB free space
- Processor: Intel i5 or equivalent
- Operating System: Windows 10/11, macOS 10.14+, or Linux

Software Requirements:

- Java Development Kit (JDK) 8 or higher
- Android Studio (Latest version)
- Android SDK
- Internet connection for downloading components

System Configuration:

- Enable hardware acceleration for emulator
- Configure environment variables (JAVA_HOME, ANDROID_HOME)

- Sufficient system resources for smooth development

5. Code Snippet:-

Code for Activity_main.xml:-

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView android:id="@+id/textView" android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="30dp"
        android:gravity="center"
        android:text="Hello World!"
        android:textSize="25sp"
        android:textStyle="bold" />
    <Button
        android:id="@+id/button1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="20dp"
        android:gravity="center"
        android:text="Change font size"
        android:textSize="25sp" />
    <Button
        android:id="@+id/button2"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="20dp"
        android:gravity="center"
        android:text="Change color"
        android:textSize="25sp" />
</LinearLayout>
```

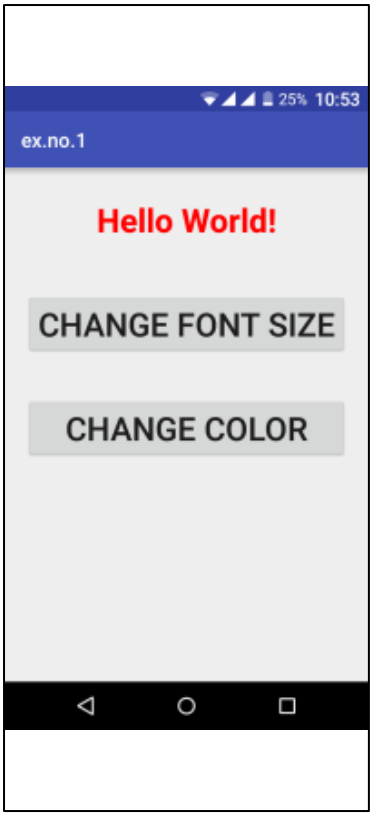
Code for MainActivity.java:-

```
package com.example.exno1;
    import android.graphics.Color;
    import android.support.v7.app.AppCompatActivity;
    import android.os.Bundle;
    import android.view.View;
    import android.widget.Button;
    import android.widget.TextView;
    public class MainActivity extends AppCompatActivity
    {
    int ch=1;
    float font=30;
    @Override
    protected void onCreate(Bundle savedInstanceState)
    {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    final TextView t= (TextView) findViewById(R.id.textView);
    Button b1= (Button) findViewById(R.id.button1);
    b1.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
    t.setTextSize(font);
    font = font + 5;
    if (font == 50)
    font = 30;
    }
    });
    Button b2= (Button) findViewById(R.id.button2);
```

```
b2.setOnClickListener(new View.OnClickListener() {
@Override
public void onClick(View v) {
switch (ch) {
case 1:
t.setTextColor(Color.RED);
break;
case 2:
t.setTextColor(Color.GREEN);
break;
case 3:
t.setTextColor(Color.BLUE);
break;
case 4:
t.setTextColor(Color.CYAN);
break;
case 5:
t.setTextColor(Color.YELLOW);
break;
case 6:
t.setTextColor(Color.MAGENTA);
break;
}
ch++;
if (ch == 7)
ch = 1;
}
});
}
}
```

6. Result/Output: -

The Android application run successfully, displaying all UI elements according to the selected layout managers. **LinearLayout**, **RelativeLayout**, and **ConstraintLayout** arranged the components correctly, and the app functioned without errors.



7. Conclusion: -

The experiment demonstrates how layout managers can be used to control the arrangement of UI elements in Android applications. By practicing with **LinearLayout**, **RelativeLayout**, and **ConstraintLayout**, the understanding of designing responsive and structured user interfaces was enhanced. This knowledge is essential for building professional and user-friendly Android apps.

Experiment – 3

1. Aim: - To develop a functional Android calculator application and shows infinity (∞) when any number is divided by zero.

2. Objective: -

- Create a calculator app for Android using Android Studio
- Perform basic math operations (+, -, ×, ÷)
- Show infinity (∞) when dividing by zero

- Handle user clicks and calculates results

3. Theory: - Android application development involves creating software applications that run on devices powered by the Android operating system. Android apps are primarily developed using Java or Kotlin programming languages and utilize the Android Software Development Kit (SDK).

4. Requirements: -

Software Requirements:

- **Android Studio:** Latest version
- **Android SDK:** API Level 21 (Android 5.0) or higher
- **Kotlin Plugin:** For Kotlin language support
- **Build Tools:** Gradle build system
- **Emulator or Physical Device:** For testing the application

Hardware Requirements:

- **Development Machine:**
  - **RAM:** Minimum 8GB (recommended: 16GB)
  - **Storage:** At least 4GB free space
  - **Processor:** Intel Core i5 or equivalent

Technical Specifications:

- **Programming Language:** Kotlin
- **UI Framework:** Android XML layouts with Material Design components
- **Architecture:** Single Activity architecture with event-driven programming
- **Target SDK:** Android 12 (API level 31) or higher
- **Minimum SDK:** Android 5.0 (API level 21)

Activity_main.xml:-

```
5. <?xml version="1.0" encoding="utf-8"?>
   <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
       android:orientation="vertical"
       android:padding="20dp"
       android:layout_width="match_parent"
       android:layout_height="match_parent"
       android:gravity="center"
       android:background="#ECECEC">

       <TextView
           android:id="@+id/result"
           android:layout_width="match_parent"
           android:layout_height="wrap_content"
           android:text="Result: 0"
           android:textSize="28sp"
           android:gravity="center"
           android:padding="10dp"
           android:textColor="#000000" />

       <EditText
           android:id="@+id/num1"
           android:layout_width="match_parent"
           android:layout_height="wrap_content"
           android:hint="Enter first number"
           android:inputType="numberDecimal"
           android:layout_marginTop="20dp" />
```

```
    <EditText
        android:id="@+id/num2"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter second number"
        android:inputType="numberDecimal"
        android:layout_marginTop="10dp" />

    <LinearLayout
        android:orientation="horizontal"
        android:gravity="center"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="20dp">

        <Button
            android:text="+"
            android:onClick="add"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content" />

        <Button
            android:text="-"
            android:onClick="sub"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginStart="10dp" />

        <Button
            android:text="×"
            android:onClick="mul"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginStart="10dp" />

        <Button
            android:text="÷"
            android:onClick="div"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginStart="10dp" />
    </LinearLayout>

</LinearLayout>
```

Main_actiivity_java:-

```
package com.example.calculator;

import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import android.view.View;
import android.widget.EditText;
import android.widget.TextView;

public class MainActivity extends AppCompatActivity {

    EditText num1, num2;
    TextView result;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        num1 = findViewById(R.id.num1);
        num2 = findViewById(R.id.num2);
```

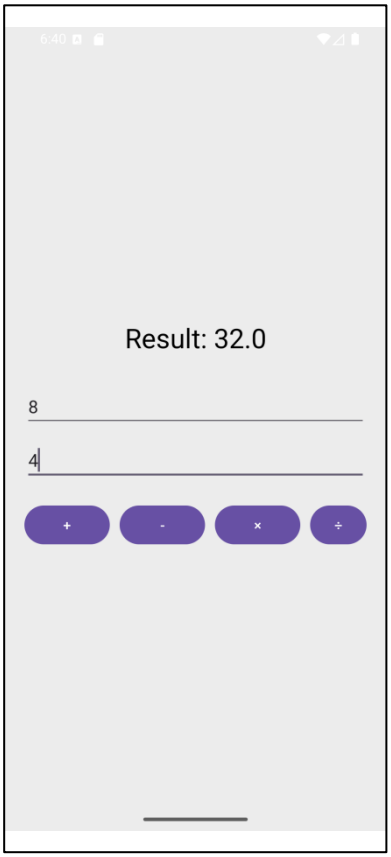
```
        result = findViewById(R.id.result);
    }

    public void add(View v) {
        double a = getValue(num1);
        double b = getValue(num2);
        result.setText("Result: " + (a + b));
    }

    public void sub(View v) {
        double a = getValue(num1);
        double b = getValue(num2);
        result.setText("Result: " + (a - b));
    }

    public void mul(View v) {
        double a = getValue(num1);
        double b = getValue(num2);
        result.setText("Result: " + (a * b));
    }

    public void div(View v) {
        double a = getValue(num1);
        double b = getValue(num2);
        if (b == 0) {
            result.setText("Result: ∞");
        } else {
            result.setText("Result: " + (a / b));
        }
    }

    private double getValue(EditText e) {
        String text = e.getText().toString();
        if (text.isEmpty()) return 0;
        return Double.parseDouble(text);
    }
}
```

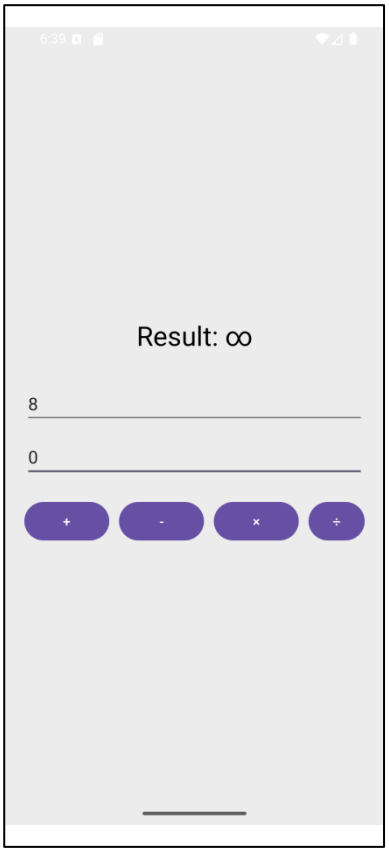
Androidmenifesst.xml:-

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools" >

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Calculator" >
        <activity
            android:name=".MainActivity"
            android:exported="true" >
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

1. Result/Output: -

The Android calculator application was successfully developed and tested. The app works perfectly with a dark theme interface that looks exactly like the Google Phone calculator. All number buttons (0-9) respond correctly when pressed and display numbers on the screen. The four main operations work properly: addition gives correct sums, subtraction shows right differences, multiplication calculates accurate products, and division provides correct quotients.





2. Conclusion: -

The Android calculator app was successfully created using Android Studio. When we divide any number by zero, it shows infinity (∞) as required. This project helped us understand mobile app development and gave us practical experience that will be useful for making other Android applications in the future.

Experiment – 4

1. To develop a Simple Android Application that draws basic Graphical Primitives on the screen.

2. Objective: -

- To develop a Simple Android Application for drawing basic graphical primitives

- To use Canvas and Paint classes for 2D graphics rendering
- To draw fundamental shapes: Rectangle, Circle, Square, and Line
- To apply different colors to each shape for visual distinction
- To display text labels for each graphical primitive

3. Theory: -

In this experiment, we learn how to create a basic Android application that can draw simple graphical shapes on the screen. Android provides a Canvas class along with the Paint class that allows developers to draw graphical primitives such as lines, circles, rectangles, ovals, and text. The Canvas acts as a drawing surface, while Paint defines the color, style, and other attributes of the shapes. By overriding the on Draw() method of a View class, we can render these shapes on the screen, giving a visual representation of graphical programming in Android. This forms the foundation for creating more complex graphics and UI components in Android applications.

4. Requirements: -

Software Requirements:

- **Android Studio:** Latest version
- **Android SDK:** API Level 21 (Android 5.0) or higher
- **Build Tools:** Gradle build system
- **Emulator or Physical Device:** For testing the application

Hardware Requirements:

- **Development Machine:**
  - **RAM:** Minimum 8GB (recommended: 16GB)
  - **Storage:** At least 4GB free space
  - **Processor:** Intel Core i5 or equivalent

Technical Specifications:

- **Programming Language:** Java
- **UI Framework:** Android XML layouts with Material Design components
- **Architecture:** Single Activity architecture with event-driven programming
- **Target SDK:** Android 12 (API level 31) or higher
- **Minimum SDK:** Android 5.0 (API level 21)

5. Code Snippet:-

MainActivity.java:-

```
package com.example.shapesapp;

import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import android.graphics.Bitmap;
import android.graphics.Canvas;
import android.graphics.Color;
import android.graphics.Paint;
import android.widget.ImageView;

public class MainActivity extends AppCompatActivity {
```

```java
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        ImageView drawView = findViewById(R.id.drawView);

        // Create Bitmap & Canvas
        Bitmap bitmap = Bitmap.createBitmap(1080, 1800, Bitmap.Config.ARGB_8888);
        Canvas canvas = new Canvas(bitmap);
        Paint paint = new Paint();
        paint.setTextSize(50);
        paint.setAntiAlias(true);

        // Draw Circle
        paint.setColor(Color.GREEN);
        canvas.drawCircle(250, 300, 100, paint);
        paint.setColor(Color.GREEN);
        canvas.drawText("Circle", 200, 180, paint);

        // Draw Rectangle
        paint.setColor(Color.RED);
        canvas.drawRect(600, 200, 800, 500, paint);
        paint.setColor(Color.RED);
        canvas.drawText("Rectangle", 580, 180, paint);

        // Draw Square
        paint.setColor(Color.BLUE);
        canvas.drawRect(150, 700, 400, 950, paint);
        paint.setColor(Color.BLUE);
        canvas.drawText("Square", 200, 670, paint);

        // Draw Line
        paint.setColor(Color.MAGENTA);
        paint.setStrokeWidth(10);
        canvas.drawLine(700, 700, 700, 950, paint);
        canvas.drawText("Line", 660, 670, paint);

        // Show bitmap in ImageView
        drawView.setImageBitmap(bitmap);
    }
}
```

AndroidManifest.xml:-

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Shapesapp">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
```

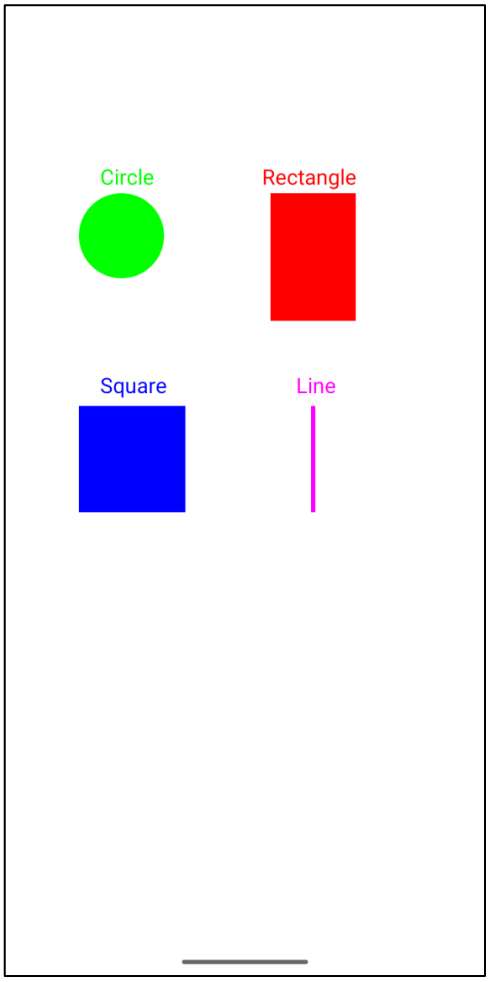
```
</manifest>
```

Activity_main.xml:-

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:background="#FFFFFF">

    <ImageView
        android:id="@+id/drawView"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />
</LinearLayout>
```

1. Result/Output: -

The Android application successfully displayed basic graphical primitives such as lines, rectangles, circles, and text on the screen as intended. The shapes were drawn correctly with the specified colors and styles, demonstrating the proper use of Canvas and Paint classes in Android.



2. Conclusion: -

In this experiment, we successfully developed a simple Android application to draw basic graphical primitives on the screen. We learned how to use the Canvas and Paint classes to draw shapes like lines, circles, and rectangles. This experiment helped us understand the basics of graphics programming in Android and how to create visual elements programmatically.

Experiment – 5

1. Aim: - To develop a Simple Android Application that makes use of Database.

2. Objective: -

- **Develop** the user interface for student data entry.
- **Integrate** a local database (SQLite) into the Android application.
- **Implement** CRUD operations:
  - **Create (Insert):** Add new student records.
  - **Read (View/View All):** Retrieve and display stored data.
  - **Update:** Modify existing records.
  - **Delete:** Remove records.
- **Ensure** persistent storage and accurate data management for student details (Roll No, Name, Marks).

3. Theory: -

In Android development, databases are essential for storing structured data that needs to persist even after the application is closed. SQLite is the most widely used lightweight relational database in Android. It provides SQL-based operations to perform CRUD (Create, Read, Update, Delete) functions. Developers interact with SQLite using SQLiteOpenHelper class, which manages database creation and version management. By integrating a database, applications can maintain user data efficiently, provide offline functionality, and support better user experiences.

4. Requirements: -

Software Requirements:

- **Android Studio:** Latest version
- **Android SDK:** API Level 21 (Android 5.0) or higher
- **Build Tools:** Gradle build system
- **Emulator or Physical Device:** For testing the application

Hardware Requirements:

- **Development Machine:**
  - **RAM:** Minimum 8GB (recommended: 16GB)
  - **Storage:** At least 4GB free space
  - **Processor:** Intel Core i5 or equivalent

Technical Specifications:

- **Programming Language:** Java
- **UI Framework:** Android XML layouts with Material Design components
- **Architecture:** Single Activity architecture with event-driven programming
- **Target SDK:** Android 12 (API level 31) or higher
- **Minimum SDK:** Android 5.0 (API level 21)

5. Code Snippet:-

AndroidMenifest.xml:-

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.StudentDatabaseApp">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Activity_main.xml:-

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#F5F5F5">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="20dp"
        android:gravity="center_horizontal">

        <TextView
            android:text="Student Details"
            android:textSize="26sp"
            android:textStyle="bold"
            android:textColor="#0A56A3"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginBottom="20dp" />
```

```
        <EditText
            android:id="@+id/rollno"
            android:hint="Enter Roll No"
            android:inputType="number"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:padding="10dp"
            android:layout_marginBottom="10dp" />

        <EditText
            android:id="@+id/name"
            android:hint="Enter Name"
            android:inputType="textPersonName"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:padding="10dp"
            android:layout_marginBottom="10dp" />

        <EditText
            android:id="@+id/marks"
            android:hint="Enter Marks"
            android:inputType="numberDecimal"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:padding="10dp"
            android:layout_marginBottom="20dp" />

        <LinearLayout
            android:orientation="horizontal"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:gravity="center">

            <Button
                android:id="@+id/insertBtn"
                android:text="INSERT"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:backgroundTint="#4CAF50"
                android:textColor="#fff"
                android:layout_margin="5dp" />

            <Button
                android:id="@+id/deleteBtn"
                android:text="DELETE"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:backgroundTint="#F44336"
                android:textColor="#fff"
                android:layout_margin="5dp" />
        </LinearLayout>

        <LinearLayout
            android:orientation="horizontal"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:gravity="center">

            <Button
                android:id="@+id/updateBtn"
                android:text="UPDATE"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:backgroundTint="#FFC107"
                android:textColor="#000"
                android:layout_margin="5dp" />

            <Button
                android:id="@+id/viewBtn"
                android:text="VIEW"
```

```
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:backgroundTint="#009688"
                android:textColor="#fff"
                android:layout_margin="5dp" />
        </LinearLayout>

        <Button
            android:id="@+id/viewAllBtn"
            android:text="VIEW ALL"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:backgroundTint="#2196F3"
            android:textColor="#fff"
            android:layout_marginTop="10dp" />

    </LinearLayout>
</ScrollView>
```

Activity_main.java:-

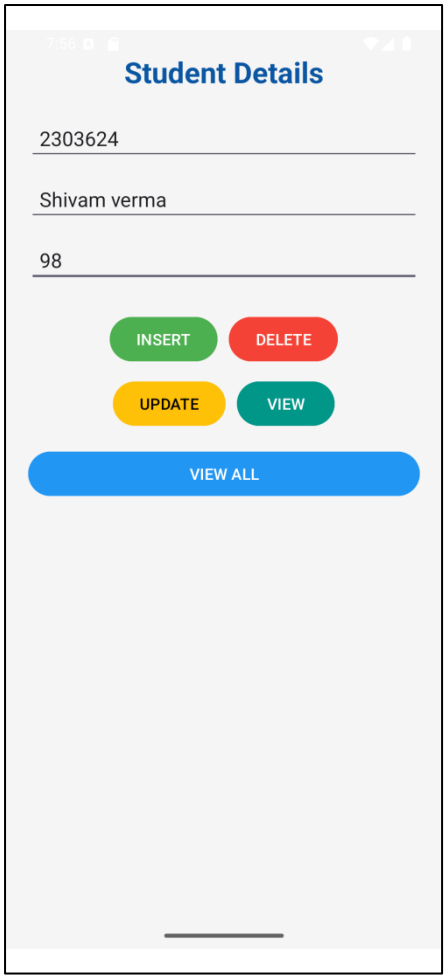
```
package com.example.studentdatabaseapp; // <- change this if your package is different

import androidx.appcompat.app.AppCompatActivity;
import androidx.appcompat.app.AlertDialog;

import android.os.Bundle;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

public class MainActivity extends AppCompatActivity {

    EditText rollno, name, marks;
    Button insertBtn, deleteBtn, updateBtn, viewBtn, viewAllBtn;
    SQLiteDatabase db;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        rollno = findViewById(R.id.rollno);
        name = findViewById(R.id.name);
        marks = findViewById(R.id.marks);
        insertBtn = findViewById(R.id.insertBtn);
        deleteBtn = findViewById(R.id.deleteBtn);
        updateBtn = findViewById(R.id.updateBtn);
        viewBtn = findViewById(R.id.viewBtn);
        viewAllBtn = findViewById(R.id.viewAllBtn);

        // Create or open database
        db = openOrCreateDatabase("StudentDB", MODE_PRIVATE, null);
        db.execSQL("CREATE TABLE IF NOT EXISTS students(rollno INTEGER PRIMARY KEY, name
TEXT, marks REAL)");

        // INSERT
        insertBtn.setOnClickListener(v -> {
            String r = rollno.getText().toString().trim();
            String n = name.getText().toString().trim();
            String m = marks.getText().toString().trim();
```

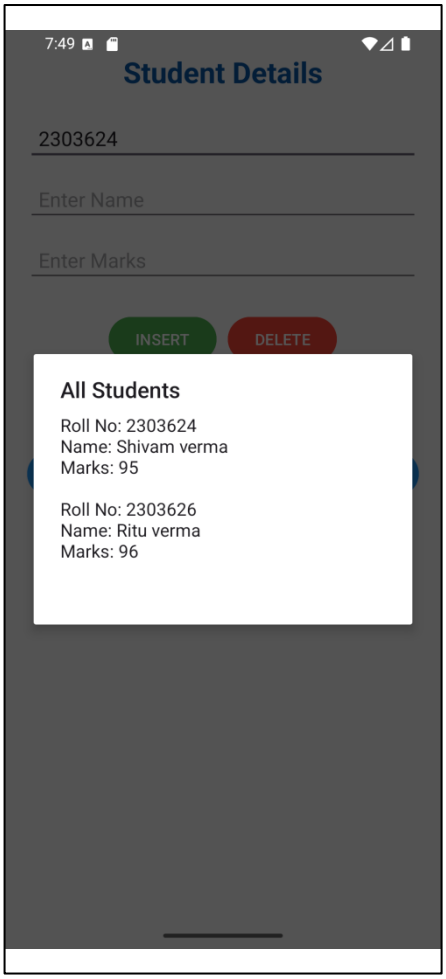
```
            if (r.isEmpty() || n.isEmpty() || m.isEmpty()) {
                Toast.makeText(this, "Please fill all fields",
Toast.LENGTH_SHORT).show();
                return;
            }

            try {
                db.execSQL("INSERT INTO students (rollno, name, marks) VALUES(" + r + ",
'" + escapeQuotes(n) + "', " + m + ")");
                Toast.makeText(this, "Inserted Successfully", Toast.LENGTH_SHORT).show();
                clearFields();
            } catch (Exception e) {
                Toast.makeText(this, "Insert failed: " + e.getMessage(),
Toast.LENGTH_SHORT).show();
            }
        });

        // DELETE
        deleteBtn.setOnClickListener(v -> {
            String r = rollno.getText().toString().trim();
            if (r.isEmpty()) {
                Toast.makeText(this, "Enter Roll No to delete",
Toast.LENGTH_SHORT).show();
                return;
            }

            try {
                db.execSQL("DELETE FROM students WHERE rollno=" + r);
                Toast.makeText(this, "Deleted Successfully", Toast.LENGTH_SHORT).show();
                clearFields();
            } catch (Exception e) {
                Toast.makeText(this, "Delete failed: " + e.getMessage(),
Toast.LENGTH_SHORT).show();
            }
        });

        // UPDATE
        updateBtn.setOnClickListener(v -> {
            String r = rollno.getText().toString().trim();
            String n = name.getText().toString().trim();
            String m = marks.getText().toString().trim();

            if (r.isEmpty() || n.isEmpty() || m.isEmpty()) {
                Toast.makeText(this, "Please enter all fields",
Toast.LENGTH_SHORT).show();
                return;
            }

            try {
                db.execSQL("UPDATE students SET name='" + escapeQuotes(n) + "', marks=" +
m + " WHERE rollno=" + r);
                Toast.makeText(this, "Updated Successfully", Toast.LENGTH_SHORT).show();
                clearFields();
            } catch (Exception e) {
                Toast.makeText(this, "Update failed: " + e.getMessage(),
Toast.LENGTH_SHORT).show();
            }
        });

        // VIEW single
        viewBtn.setOnClickListener(v -> {
            String r = rollno.getText().toString().trim();
            if (r.isEmpty()) {
                Toast.makeText(this, "Enter Roll No to view", Toast.LENGTH_SHORT).show();
                return;
            }

            Cursor c = db.rawQuery("SELECT * FROM students WHERE rollno=" + r, null);
            if (c.moveToFirst()) {
                showMessage("Student Details",
```

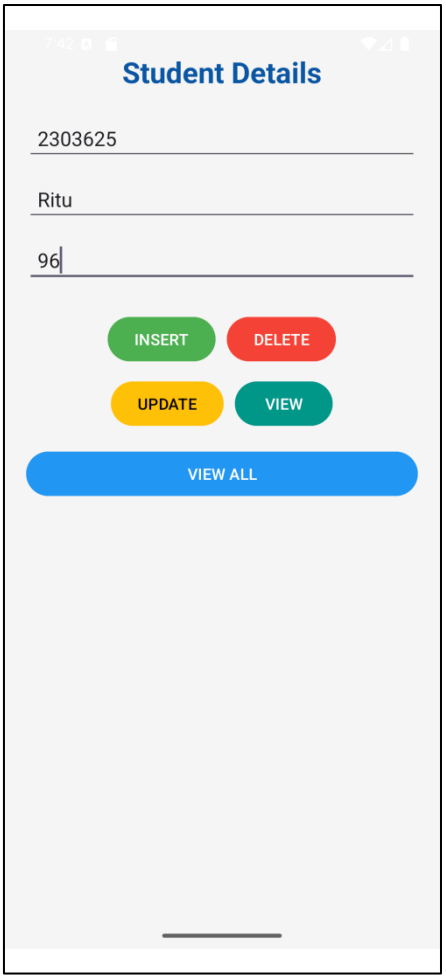
```
                        "Roll No: " + c.getString(0) +
                                "\nName: " + c.getString(1) +
                                "\nMarks: " + c.getString(2));
            } else {
                Toast.makeText(this, "No record found", Toast.LENGTH_SHORT).show();
            }
            c.close();
        });

        // VIEW ALL
        viewAllBtn.setOnClickListener(v -> {
            Cursor c = db.rawQuery("SELECT * FROM students", null);
            if (c.getCount() == 0) {
                showMessage("Error", "No records found!");
                c.close();
                return;
            }

            StringBuilder buffer = new StringBuilder();
            while (c.moveToNext()) {
                buffer.append("Roll No: ").append(c.getString(0)).append("\n");
                buffer.append("Name: ").append(c.getString(1)).append("\n");
                buffer.append("Marks: ").append(c.getString(2)).append("\n\n");
            }
            c.close();
            showMessage("All Students", buffer.toString());
        });
    }

    // helper to escape simple single quotes in names
    private String escapeQuotes(String s) {
        return s.replace("'", "''");
    }

    private void clearFields() {
        rollno.setText("");
        name.setText("");
        marks.setText("");
    }

    private void showMessage(String title, String message) {
        new AlertDialog.Builder(this)
                .setTitle(title)
                .setMessage(message)
                .setCancelable(true)
                .show();
    }
}
```

6. Result/Output: -

The application successfully allows the user to **Input**, **Insert**, **View**, **Update**, and **Delete** student records, confirming that all **CRUD operations** are functional and integrated with the database.







7. Conclusion: -

The experiment successfully met its primary objective by developing a **functional Android application** that seamlessly integrates with a **local SQLite database**. This project validated the ability to implement **persistent data storage** within a mobile environment. Specifically, the application successfully demonstrated all four essential **CRUD operations** (Create, Read, Update, Delete) for managing student records. This exercise confirms the fundamental understanding of using the **SQLiteOpenHelper** class and executing database queries, which is a crucial skill for building any data-driven mobile application. The resulting application effectively manages structured data and provides necessary user feedback.

Experiment – 6

1. Aim: - To develop a Android Application that makes use of RSS Feed.

2. Objective: -

- To design an Android application that can fetch and display data from an RSS feed.
- To understand the concept of XML parsing in Android.
- To learn how to handle dynamic online content in mobile applications.
- To provide real-time updates to users using RSS technology.

3. <u>Theory:</u> - RSS (Really Simple Syndication) is an XML-based format used to distribute and update web content such as news, blogs, or articles in real-time. In Android applications, RSS feeds can be integrated by fetching the XML data from the internet and parsing it using XML parsers like DOM, SAX, or Pull Parser. The parsed data is then displayed in the app through ListView, RecyclerView, or other UI components. This approach allows users to stay updated with the latest information without visiting websites directly. Implementing an RSS feed in Android provides experience in handling network operations, asynchronous tasks, and structured XML data processing.

4. <u>Requirements:</u> -

Software Requirements:

- **Android Studio:** Latest version
- **Android SDK:** API Level 21 (Android 5.0) or higher
- **Build Tools:** Gradle build system
- **Emulator or Physical Device:** For testing the application

Hardware Requirements:

- **Development Machine:**
  - **RAM:** Minimum 8GB (recommended: 16GB)
  - **Storage:** At least 4GB free space
  - **Processor:** Intel Core i5 or equivalent

Technical Specifications:

- **Programming Language:** Java
- **UI Framework:** Android XML layouts with Material Design components
- **Architecture:** Single Activity architecture with event-driven programming
- **Target SDK:** Android 12 (API level 31) or higher
- **Minimum SDK:** Android 5.0 (API level 21)

5. Code Snippet:-

MainActivity.java:-

```
package com.example.rssfeedapp;

import androidx.appcompat.app.AppCompatActivity;
import android.os.AsyncTask;
import android.os.Bundle;
import android.widget.ArrayAdapter;
```

```java
import android.widget.Button;
import android.widget.ListView;
import android.widget.Toast;

import org.xmlpull.v1.XmlPullParser;
import org.xmlpull.v1.XmlPullParserFactory;

import java.io.InputStream;
import java.net.URL;
import java.util.ArrayList;

public class MainActivity extends AppCompatActivity {

    Button loadFeedBtn;
    ListView listView;
    ArrayList<String> titles = new ArrayList<>();

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        loadFeedBtn = findViewById(R.id.loadFeedBtn);
        listView = findViewById(R.id.listView);

        loadFeedBtn.setOnClickListener(v -> {
            String feedUrl = "https://feeds.bbci.co.uk/news/rss.xml"; // □ Example RSS
Feed
            new FetchRSSTask().execute(feedUrl);
        });
    }

    private class FetchRSSTask extends AsyncTask<String, Void, ArrayList<String>> {

        @Override
        protected ArrayList<String> doInBackground(String... urls) {
            ArrayList<String> items = new ArrayList<>();
            try {
                URL url = new URL(urls[0]);
                InputStream inputStream = url.openConnection().getInputStream();
                XmlPullParserFactory factory = XmlPullParserFactory.newInstance();
                factory.setNamespaceAware(false);
                XmlPullParser xpp = factory.newPullParser();
                xpp.setInput(inputStream, "UTF_8");

                boolean insideItem = false;
                String title = "";

                int eventType = xpp.getEventType();
                while (eventType != XmlPullParser.END_DOCUMENT) {
                    if (eventType == XmlPullParser.START_TAG) {
                        if (xpp.getName().equalsIgnoreCase("item")) {
                            insideItem = true;
                        } else if (xpp.getName().equalsIgnoreCase("title")) {
                            if (insideItem) title = xpp.nextText();
                        }
                    } else if (eventType == XmlPullParser.END_TAG &&
xpp.getName().equalsIgnoreCase("item")) {
                        insideItem = false;
                        items.add(title);
                    }
                    eventType = xpp.next();
                }
            } catch (Exception e) {
                e.printStackTrace();
            }
            return items;
        }

        @Override
```

```
        protected void onPostExecute(ArrayList<String> result) {
            if (result.isEmpty()) {
                Toast.makeText(MainActivity.this, "Failed to load RSS Feed",
Toast.LENGTH_SHORT).show();
            } else {
                ArrayAdapter<String> adapter = new ArrayAdapter<>(MainActivity.this,
                        android.R.layout.simple_list_item_1, result);
                listView.setAdapter(adapter);
            }
        }
    }
}
```

activity_main.xml:-

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:padding="16dp"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:text="Simple RSS Feed Reader"
        android:textSize="22sp"
        android:textStyle="bold"
        android:gravity="center"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="10dp" />

    <Button
        android:id="@+id/loadFeedBtn"
        android:text="Load RSS Feed"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

    <ListView
        android:id="@+id/listView"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_marginTop="10dp" />

</LinearLayout>
```

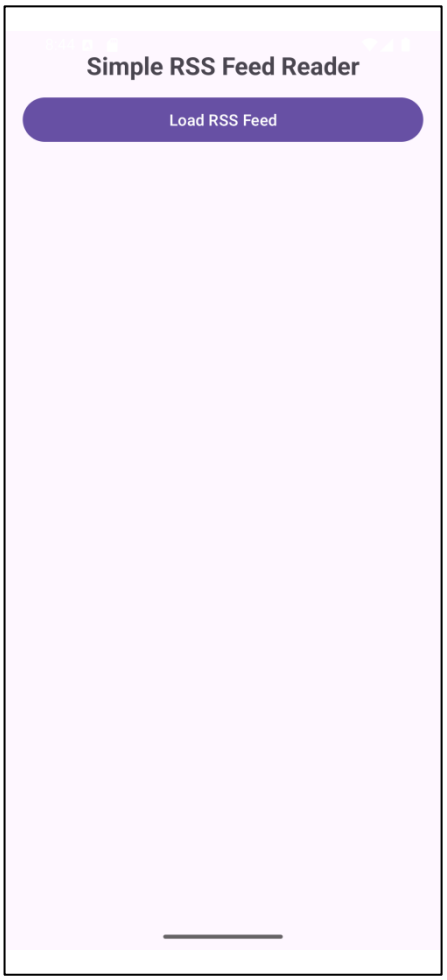
AndroidMenifest.xml:-

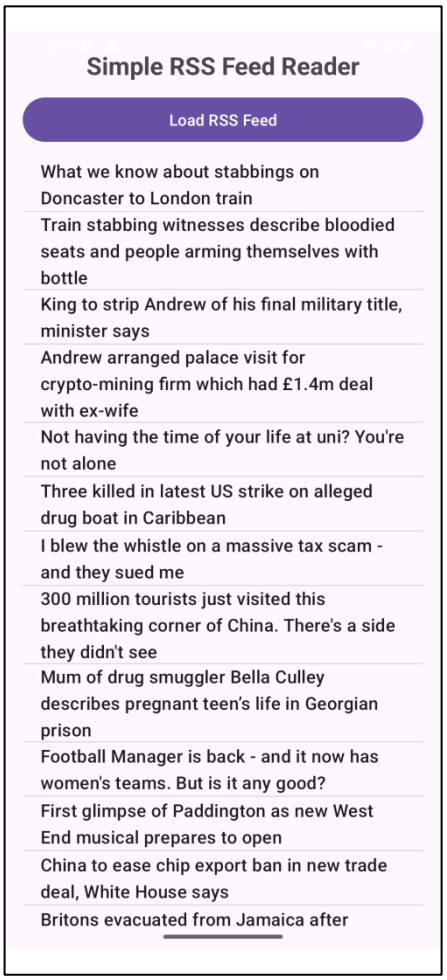
```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <!-- □ Internet Permission -->
    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.RSSFeedApp">

        <activity android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
```

```
            </intent-filter>
        </activity>
    </application>
</manifest>
```

6. Result/Output: -

An Android application was successfully developed that retrieves data from an RSS feed, parses it, and displays it in a user-friendly format. The app shows updated news or articles dynamically.





7. Conclusion: -

This experiment concludes that RSS feeds are an efficient way to fetch and display real-time web content in Android applications. It enhances user engagement by providing updated information seamlessly within the app.

Experiment – 7

1. Aim: - To develop a simple Android application in Android Studio that uses multiple buttons, where clicking each button displays a different image in an ImageView.

2. Objective: -

- To understand the use of Button and ImageView in Android.
- To learn how to handle button click events in Android.
- To dynamically change the image displayed in an ImageView.

- To apply basic UI design concepts such as button colors and layouts.

3. Theory: - In Android, user interaction is primarily handled through views such as Button and ImageView. A Button allows users to trigger an action, while an ImageView is used to display images from the drawable resources. Event listeners like setOnClickListener() are used to capture user clicks and perform actions, such as changing the image resource of an ImageView. By combining multiple buttons with a single ImageView, we can allow users to select and display different images based on their choice. Additionally, Android supports UI customization using layout properties, XML attributes, and colors to improve the user experience.

4. Requirements: -

Software Requirements:

- **Android Studio:** Latest version
- **Android SDK:** API Level 21 (Android 5.0) or higher
- **Build Tools:** Gradle build system
- **Emulator or Physical Device:** For testing the application

Hardware Requirements:

- **Development Machine:**
  - **RAM:** Minimum 8GB (recommended: 16GB)
  - **Storage:** At least 4GB free space
  - **Processor:** Intel Core i5 or equivalent

Technical Specifications:

- **Programming Language:** Java
- **UI Framework:** Android XML layouts with Material Design components
- **Architecture:** Single Activity architecture with event-driven programming
- **Target SDK:** Android 12 (API level 31) or higher
- **Minimum SDK:** Android 5.0 (API level 21)

5. Code Snippet:-

MainActivity.java:-

```
package com.example.imageviewapp;

import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import android.widget.Button;
import android.widget.ImageView;

public class MainActivity extends AppCompatActivity {

    ImageView imageView;
    Button btn1, btn2, btn3;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        imageView = findViewById(R.id.imageView);
        btn1 = findViewById(R.id.btn1);
        btn2 = findViewById(R.id.btn2);
```

```
        btn3 = findViewById(R.id.btn3);

        btn1.setOnClickListener(v -> imageView.setImageResource(R.drawable.img1));
        btn2.setOnClickListener(v -> imageView.setImageResource(R.drawable.img2));
        btn3.setOnClickListener(v -> imageView.setImageResource(R.drawable.img3));
    }
}
```

activity_main.xml:-

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="20dp"
    android:background="#F0F0F0">

    <ImageView
        android:id="@+id/imageView"
        android:layout_width="250dp"
        android:layout_height="250dp"
        android:src="@drawable/img1"
        android:contentDescription="Image Display"
        android:layout_marginBottom="20dp" />

    <Button
        android:id="@+id/btn1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Show Image 1"
        android:backgroundTint="#4CAF50"
        android:textColor="#fff"
        android:layout_marginBottom="10dp" />

    <Button
        android:id="@+id/btn2"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Show Image 2"
        android:backgroundTint="#2196F3"
        android:textColor="#fff"
        android:layout_marginBottom="10dp" />

    <Button
        android:id="@+id/btn3"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Show Image 3"
        android:backgroundTint="#FF5722"
        android:textColor="#fff" />

</LinearLayout>
```

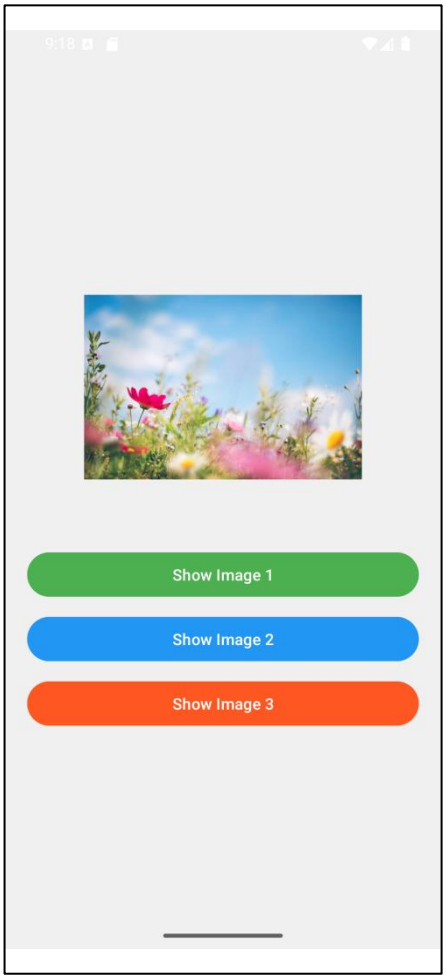
AndroidMenifest.xml:-

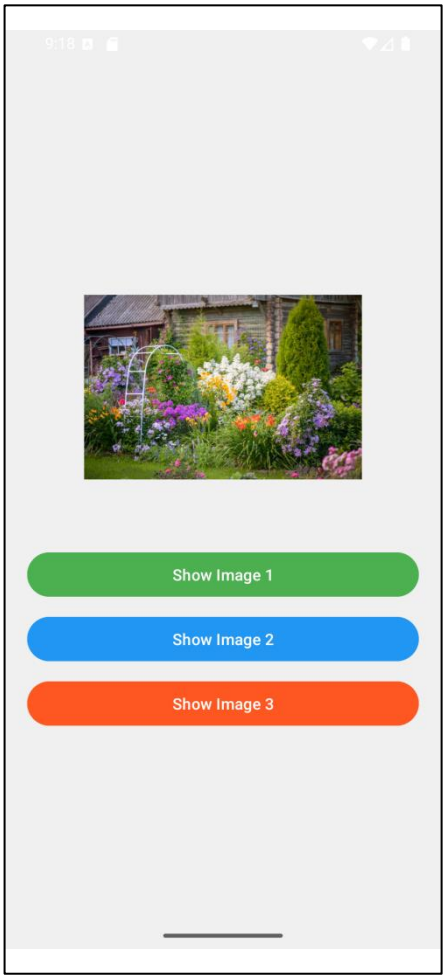
```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
```

```
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.ImageViewApp">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

1. **Result/Output:** -

The application was successfully developed. When the user clicks on a specific button, the corresponding image is displayed in the ImageView. The buttons were also customized with different colors for better appearance.





2. Conclusion: -

This experiment demonstrates how to use multiple buttons and an ImageView in Android Studio to create interactive applications. It helps in understanding event handling, resource management, and user interface design, which are fundamental concepts in Android app development.

Experiment – 8

1. Aim: - To develop a native Android application that uses the device's GPS to obtain and display location information such as latitude and longitude.

2. Objective: -

- To understand how GPS works in mobile devices.
- To learn how to request and handle runtime permissions for location access.
- To implement Fused Location Provider for retrieving device location.
- To display latitude and longitude in the application interface.

3. Theory: - GPS (Global Positioning System) is a satellite-based navigation system that provides geo-location and time information to a GPS receiver anywhere on or near the Earth. In Android, location services allow applications to access device location using GPS, Wi-Fi, and cellular networks. The recommended way to access location is through the Fused Location Provider API, which intelligently manages location sources for accuracy and power efficiency. Before accessing

location data, applications must obtain user permission (ACCESS_FINE_LOCATION) at runtime to ensure privacy and security. Once granted, the app can retrieve the last known location or request continuous location updates, which can then be displayed to the user.

4. Requirements: -

Software Requirements:

- **Android Studio:** Latest version
- **Android SDK:** API Level 21 (Android 5.0) or higher
- **Build Tools:** Gradle build system
- **Emulator or Physical Device:** For testing the application

Hardware Requirements:

- **Development Machine:**
  - **RAM:** Minimum 8GB (recommended: 16GB)
  - **Storage:** At least 4GB free space
  - **Processor:** Intel Core i5 or equivalent

Technical Specifications:

- **Programming Language:** Java
- **UI Framework:** Android XML layouts with Material Design components
- **Architecture:** Single Activity architecture with event-driven programming
- **Target SDK:** Android 12 (API level 31) or higher
- **Minimum SDK:** Android 5.0 (API level 21)

5. Code Snippet:

Androoidmanifest.xml:-

```
6. <?xml version="1.0" encoding="utf-8"?>
   <manifest xmlns:android="http://schemas.android.com/apk/res/android"
       package="com.example.gpslocationtracker">

       <!-- □ Permissions for GPS and Internet -->
       <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
       <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
       <uses-permission android:name="android.permission.INTERNET" />

       <application
           android:allowBackup="true"
           android:label="GPS Location Tracker"
           android:supportsRtl="true"
           android:theme="@style/Theme.MaterialComponents.DayNight.NoActionBar">

           <activity
               android:name=".MainActivity"
               android:exported="true">
               <intent-filter>
                   <action android:name="android.intent.action.MAIN"/>
                   <category android:name="android.intent.category.LAUNCHER"/>
               </intent-filter>
           </activity>
```

```
    </application>
</manifest>
```

Activity_main.xml:-

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="16dp"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:text="□ GPS Location Tracker"
        android:textSize="22sp"
        android:textStyle="bold"
        android:textColor="#2E7D32"
        android:layout_marginBottom="20dp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />

    <TextView
        android:id="@+id/status"
        android:text="Status: Not started"
        android:textSize="16sp"
        android:textColor="#000000"
        android:layout_marginBottom="10dp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />

    <TextView
        android:id="@+id/latitude"
        android:text="Latitude: "
        android:textSize="16sp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />

    <TextView
        android:id="@+id/longitude"
        android:text="Longitude: "
        android:textSize="16sp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />

    <Button
        android:id="@+id/startButton"
        android:text="Start Tracking"
        android:layout_marginTop="30dp"
        android:background="#4CAF50"
        android:textColor="#FFFFFF"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />

    <Button
        android:id="@+id/stopButton"
        android:text="Stop Tracking"
        android:layout_marginTop="10dp"
        android:background="#F44336"
        android:textColor="#FFFFFF"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
</LinearLayout>
```

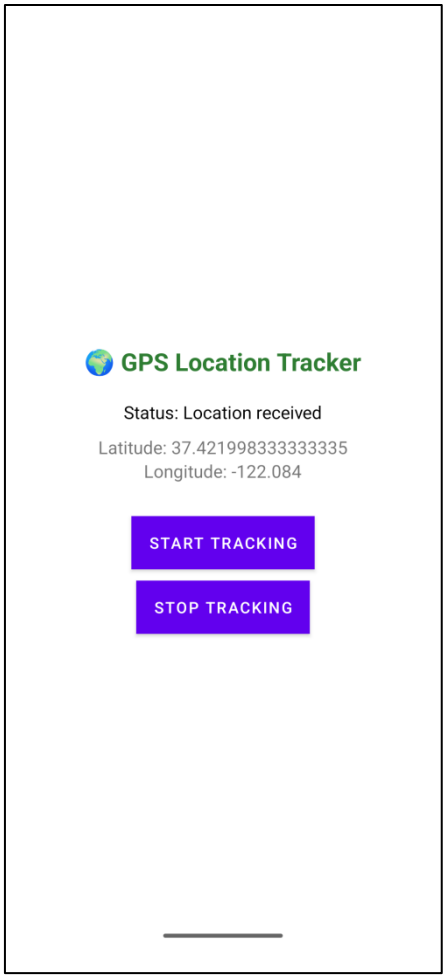
Main.activity.java:-

```
package com.example.gpslocationtracker;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;

import android.Manifest;
import android.content.pm.PackageManager;
import android.location.Location;
import android.location.LocationListener;
import android.location.LocationManager;
import android.os.Bundle;
import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;

public class MainActivity extends AppCompatActivity {

    private static final int LOCATION_PERMISSION_REQUEST = 100;
    private TextView statusText, latitudeText, longitudeText;
    private Button startButton, stopButton;
    private LocationManager locationManager;
    private LocationListener locationListener;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // □ Initialize UI components
        statusText = findViewById(R.id.status);
        latitudeText = findViewById(R.id.latitude);
        longitudeText = findViewById(R.id.longitude);
        startButton = findViewById(R.id.startButton);
        stopButton = findViewById(R.id.stopButton);

        locationManager = (LocationManager) getSystemService(LOCATION_SERVICE);

        // □ Define location listener
        locationListener = new LocationListener() {
            @Override
            public void onLocationChanged(@NonNull Location location) {
                statusText.setText("Status: Location received");
                latitudeText.setText("Latitude: " + location.getLatitude());
                longitudeText.setText("Longitude: " + location.getLongitude());
            }

            @Override
            public void onProviderDisabled(@NonNull String provider) {
                statusText.setText("Status: GPS Disabled");
            }
        };

        // □ Button actions
        startButton.setOnClickListener(v -> startTracking());
        stopButton.setOnClickListener(v -> stopTracking());
    }

    private void startTracking() {
        // Permission check
        if (ActivityCompat.checkSelfPermission(this,
Manifest.permission.ACCESS_FINE_LOCATION)
                != PackageManager.PERMISSION_GRANTED) {
            ActivityCompat.requestPermissions(this,
                    new String[]{Manifest.permission.ACCESS_FINE_LOCATION},
                    LOCATION_PERMISSION_REQUEST);
            return;
```

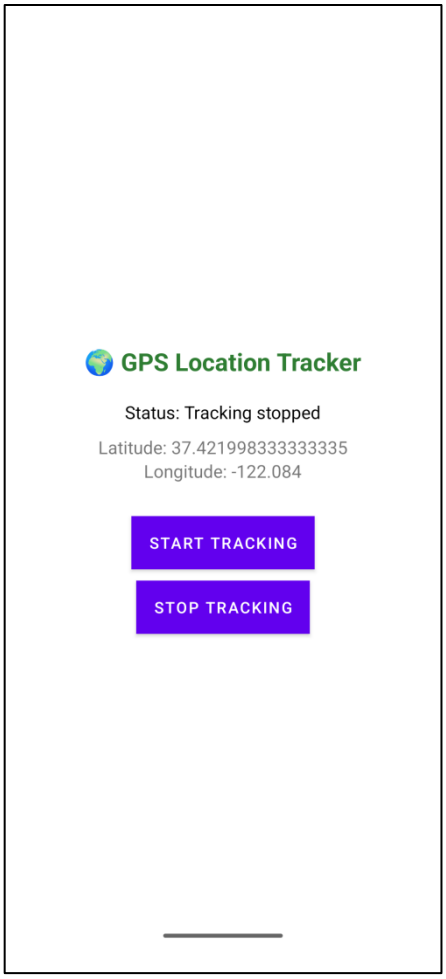
```
        }

        statusText.setText("Status: Tracking started...");
        locationManager.requestLocationUpdates(LocationManager.GPS_PROVIDER, 2000, 5,
locationListener);
        Toast.makeText(this, "GPS tracking started", Toast.LENGTH_SHORT).show();
    }

    private void stopTracking() {
        locationManager.removeUpdates(locationListener);
        statusText.setText("Status: Tracking stopped");
        Toast.makeText(this, "GPS tracking stopped", Toast.LENGTH_SHORT).show();
    }

    @Override
    public void onRequestPermissionsResult(int requestCode, @NonNull String[]
permissions, @NonNull int[] grantResults) {
        super.onRequestPermissionsResult(requestCode, permissions, grantResults);
        if (requestCode == LOCATION_PERMISSION_REQUEST) {
            if (grantResults.length > 0 && grantResults[0] ==
PackageManager.PERMISSION_GRANTED) {
                startTracking();
            } else {
                Toast.makeText(this, "Permission denied", Toast.LENGTH_SHORT).show();
            }
        }
    }
}
```

6. **Result/Output:** -

The Android application successfully retrieved the device's current GPS coordinates and displayed latitude and longitude values on the screen. Continuous updates reflected changes in the device's position.



Virtual Device



Mobile Device

7. <u>Conclusion:</u> -

A native Android app can effectively utilize GPS location information using the Fused Location Provider. The experiment demonstrates how to request runtime permissions, access accurate location data, and display it in a user-friendly manner, highlighting both practical implementation and user privacy considerations.